

5 - LLM in DevOps

Name of report: LLM_LAB_5_Nikita_Niakhai

Course: DevOps

Performed by *Nikita Niakhai*

Date submission: 06.03.2026



In this lab, students explore how Large Language Models (LLMs) can be used as assistive tools in DevOps environments. The lab focuses on integrating an LLM into an existing CI/CD pipeline with Kubernetes deployment and security checks, and evaluating how LLMs can support explanation, troubleshooting, and security-aware decision-making.

Prerequisites: Virtualization, Containerization (Docker), CI/CD pipelines, Basic Linux and Git knowledge.

Students will reuse: the CI/CD pipeline from Lab 3, a containerized application from previous labs, and a Kubernetes deployment configuration.

Task 1 — CI/CD Pipeline Baseline with Security and Kubernetes

Before introducing the LLM, you will prepare a secure DevOps baseline.

1. Show the CI/CD pipeline you implemented in Lab 3, including the following stages:
 - Source code checkout
 - Build
 - Test
 - Container image build and deploy

2. Add a Static Application Security Testing (SAST) stage to your pipeline (using tools like Bandit, **Semgrep**, Sonarqube). Show the output of the security scan.
3. Extend your pipeline to include a deployment stage to Kubernetes.

Execution

The pipeline from Lab 3 covered five stages: **build** → **test** → **docker-build** → **docker-push** → **deploy**. The `build-app` job validates the repository contents and saves artifacts. `test-app` verifies that `index.html` is present. `build-docker` and `push-docker` build and push the image to Docker Hub. The original `deploy-app` job SSHed into a VM and ran the container with `docker run`.

Hosts configuration is done on all 3 VMs, here is example for VM1:

```
sudo nano /etc/hosts
```

```
\\ \\
```

Add these lines:

```
\\ \\
```

```
192.168.30.101    st19.sne.com gitlab
192.168.30.102    runner.st19.sne.com runner
192.168.30.103    deploy.st19.sne.com deploy
```

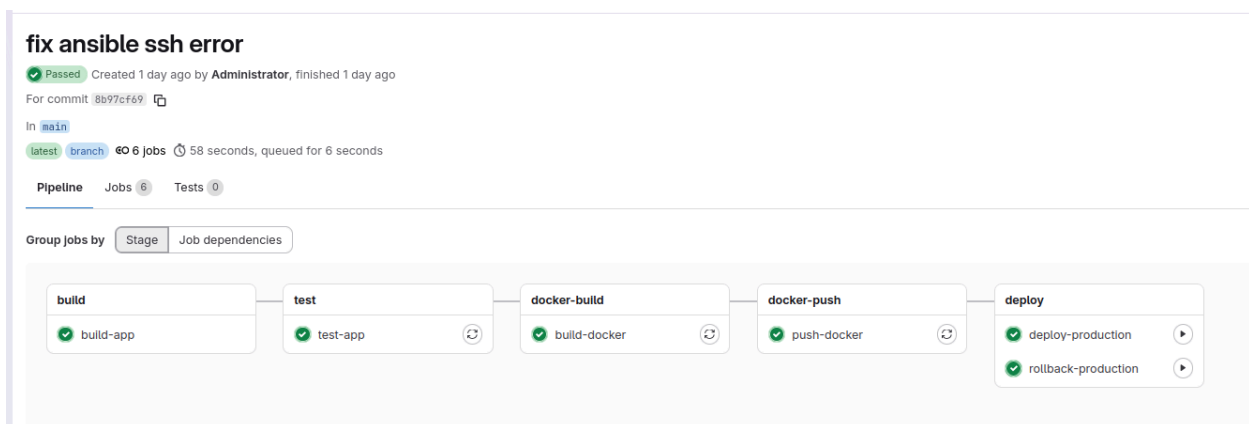


Figure 1.1 — CI/CD pipeline overview from Lab 3

For Lab 5 the following changes were made:

- A new `sast` stage was inserted after `test`, running **Semgrep** (`semgrep scan --config=auto -json`). `allow_failure: true` is set so a finding does not block the rest of the pipeline. The JSON report is saved as an artifact.
- The old SSH-based `deploy-app` job was replaced with `deploy-k8s`, which decodes `$KUBECONFIG_B64`, runs `kubectl apply -f k8s/`, and verifies the rollout. This deploys the same Docker image to a Kubernetes cluster.
- Two new parallel `llm-review` stage jobs were added (described in Task 2).



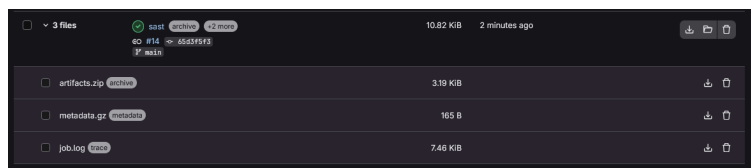
Check `.lab3_gitlab-ci.yml` file for lab 3 configurations provided in the folder

```
Administrator / st19-repo / Jobs / #24
Log timestamps in UTC. Search visible log output
11 17:41:34 Checking out v3.0.0 as detached HEAD (ref is main)...
12 17:41:34 Skipping Git submodules setup
13 17:41:34 Downloading artifacts
14 17:41:35 Downloading artifacts for build-app (22)...
15 17:41:35 Runtime platform arch=amd64 os=Linux pid=16860 revision=9ff
b4ae0 version=18.8.0
16 17:41:35 Downloading artifacts from coordinator... ok correlation_id=81KJZHEBQ8XCHQWKF78Y4GC117
host=st19.sne.com id=22 responseStatus=200 OK token=64_8L3xi6
17 17:41:35 WARNING: Part of .git directory is on the list of files to extract
18 17:41:35 WARNING: This may introduce unexpected problems
19 17:41:35 Executing "step_script" stage of the job script
20 17:41:35 $ echo "Running SAST with Semgrep..."
21 17:41:35 Running SAST with Semgrep...
22 17:41:35 $ python3 -m venv /tmp/sast-venv
23 17:41:39 $ /tmp/sast-venv/bin/pip install semgrep --quiet
24 17:44:24 $ /tmp/sast-venv/bin/semgrep scan --config=auto --json -o semgrep-report.json . || true
25 17:44:35
26 17:44:35
27 17:44:35
28 17:44:35 Scan Status
29 17:44:35
30 17:44:35 Scanning 16 files tracked by git with 1064 Code rules:
31 17:44:35
32 17:44:35
33 17:44:35
34 17:44:35
35 17:44:35
36 17:44:35
37 17:44:35
38 17:44:35
39 17:44:35
40 17:44:35
41 17:44:35
42 17:44:35
...
```

Language	Rules	Files	Origin	Rules
<multilang>	61	16	Community	1064
python	243	4		
yaml	31	3		
js	156	2		
dockerfile	6	1		
json	4	1		
bash	4	1		
html	1	1		

```
✓ Scan completed successfully.
• Findings: 6 (6 blocking)
• Rules run: 501
• Targets scanned: 16
• Parsed lines: ~99.6%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 501 rules on 16 files: 6 findings.
$ echo "SAST scan completed"
SAST scan completed
Uploading artifacts for successful job
Uploading artifacts...
Runtime platform                                arch=amd64 os=linux pid=16977 revision=9ff
b4aa0 version=18.8.0
semgrep-report.json: found 1 matching artifact files and directories
Uploading artifacts as "archive" to coordinator... 201 Created correlation_id=01KJZHNYSQ9FV
YSS9ZJB5FY6 id=24 responseStatus=201 Created token=64_8L3xiG
Cleaning up project directory and file based variables
Job succeeded
```

Figure 1.2 — SAST stage added to pipeline (result of a stage completed)



The screenshot shows a file explorer interface with a table of files. At the top, there is a summary bar for a folder named 'sast' (containing 12 more files), with a size of 10.82 KB and a timestamp of '2 minutes ago'. The table lists three files: 'artifacts.zip' (319 KiB), 'metadata.gz' (165 B), and 'job.log' (7.46 KiB). Each file has a checkbox on the left and download/upload icons on the right.

File Name	Size
artifacts.zip	319 KiB
metadata.gz	165 B
job.log	7.46 KiB

Figure 1.3 — SAST tool output: artifacts, content of a report (Semgrep)

```

EGA > SNE-25 > 5_DevOps > 1_Labs > 5_LLM_in_DevOps > upload_moodle > semgrep-report.json > results
{
  "version": "1.154.0",
  "results": [ ... ],
  "errors": [ ... ],
  "paths": {
    "scanned": [ ... ]
  },
  "time": {
    "rules": [],
    "rules_parse_time": 3.286947011947632,
    "profiling_times": { ... },
    "parsing_time": { ... },
    "scanning_time": {
      "total_time": 201.82221603393555,
      "per_file_time": { ... },
      "very_slow_stats": { ... },
      "very_slow_files": [
        { "fpath": ".gitlab-ci.yml", "ftime": 5.648835897445679 },
        { "fpath": "scripts/llm_analyze_failure.py", "ftime": 64.62798380851746 },
        { "fpath": "script.js", "ftime": 65.12696719169617 },
        { ... }
      ]
    },
    "matching_time": { ... },
    "tainting_time": { ... },
    "fixpoint_timeouts": [],
    "prefiltering": { ... },
    "targets": [],
    "total_bytes": 0,
    "max_memory_bytes": 1290282560
  },
  "engine_requested": "OSS",

```

Figure 1.3.1 — SAST tool output: all content of the report (Semgrep)

```

semgrep-report.json
Y: > MEGA > SNE-25 > 5_DevOps > 1_Labs > 5_LLM_in_DevOps > upload_moodle > {} semgrep-report.json > {} results > {} 0 > {} extra
1
2 {
3   "version": "1.154.0",
4   "results": [
5     {
6       "check_id": "dockerfile.security.missing-user.missing-user",
7       "path": "Dockerfile",
8       "start": { "line": 7, "col": 1, "offset": 60 },
9       "end": { "line": 7, "col": 35, "offset": 94 },
10      "extra": {
11        "message": "By not specifying a USER, a program in the container may run as 'root'. This is a security issue in the container. Ensure that the last USER in a Dockerfile is a USER other than 'root'.",
12        "fix": "USER non-root\nCMD [\"nginx\", \"-g\", \"daemon off;\"]",
13        "metadata": {
14          "cwe": ["CWE-250: Execution with Unnecessary Privileges"],
15          "category": "security",
16          "technology": ["dockerfile"],
17          "confidence": "MEDIUM",
18          "owasp": ["A04:2021 - Insecure Design", "A06:2025 - Insecure Design"],
19          "references": ["https://owasp.org/Top10/A04_2021-Insecure_Design"],
20          "subcategory": ["audit"],
21          "likelihood": "LOW",
22          "impact": "MEDIUM",
23          "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
24          "vulnerability_class": ["Improper Authorization"],
25          "source": "https://semgrep.dev/r/dockerfile.security.missing-user.missing-user",
26          "shortlink": "https://sg.run/Gbvn"
27        },
28        "severity": "ERROR",
29        "fingerprint": "requires login",
30        "lines": "requires login",
31        "validation_state": "NO_VALIDATOR",
32        "engine_kind": "OSS"
33      }
34    ]
35  }

```

Figure 1.3.2 — SAST tool output: result fields in the report (Semgrep)

K3S installation steps

```

# 1. Install k3s
curl -sfL https://get.k3s.io | sh -

# 2. Verify
sudo k3s kubectl get nodes

# 3. Copy config
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config

# 4. Now kubectl works
kubectl cluster-info
kubectl get nodes

# 5. Access policy
sudo mkdir -p /home/gitlab-runner/.kube
sudo cp /etc/rancher/k3s/k3s.yaml /home/gitlab-runner/.kube/c

```

```
onfig
sudo chown -R gitlab-runner:gitlab-runner /home/gitlab-runne
r/.kube
```

K8S PIPELINE STAGE

```
# CD Stage: Deploy to Kubernetes
deploy-k8s:
  stage: deploy
  tags:
    - st19-runner
  dependencies: []
  script:
    - echo "Deploying to Kubernetes..."
    - curl -LO "https://dl.k8s.io/release/$(curl -Ls https://
dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
    - chmod +x kubectl
    - mkdir -p $HOME/bin/
    - mv kubectl $HOME/bin/
    - export PATH="$HOME/bin:$PATH"
    - mkdir -p ~/.kube
    - echo "$KUBECONFIG_B64" | base64 -d > ~/.kube/config
    - kubectl apply -f k8s/
    - kubectl rollout status deployment/simple-pwa --timeout=
120s
    - echo "Kubernetes deployment completed successfully!"
  only:
    - main
```

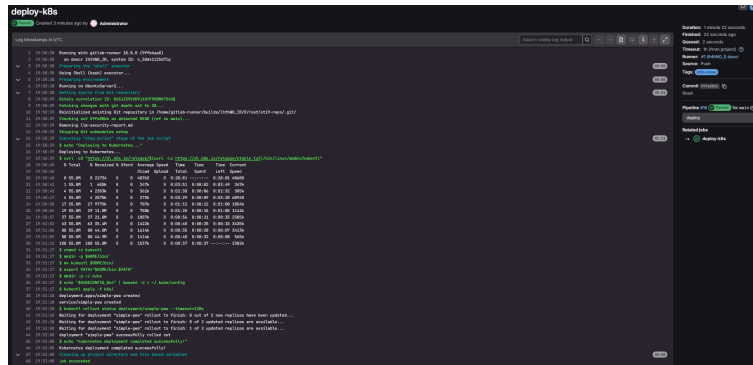


Figure 1.4 — Kubernetes deployment stage in pipeline

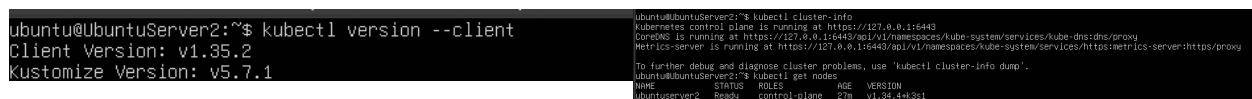


Figure 1.4.1 — Kubernetes and k3s installation

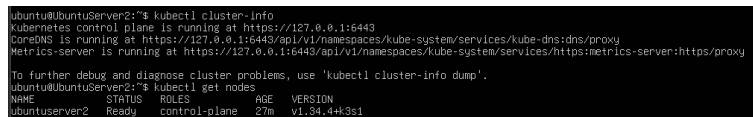


Figure 1.4.2 — Kubernetes and k3s finally running on VM2 runner machine



Figure 1.4.3 — K8S configuration environment variable added to CI/CD

Task 2 — LLM Setup and Deployment

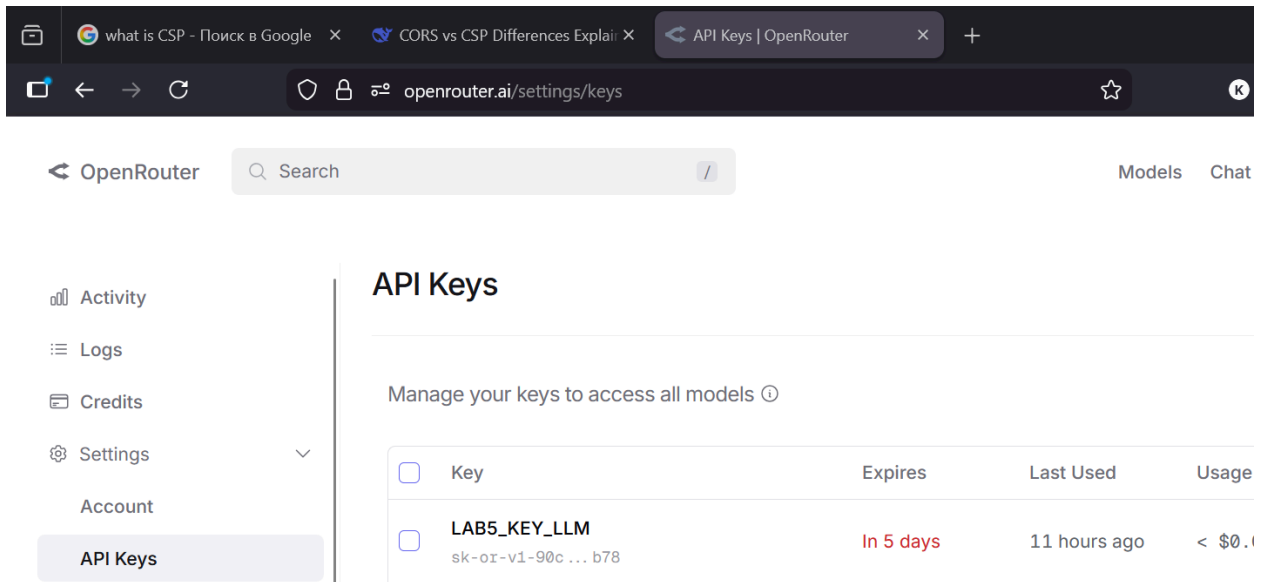
In this task, you will prepare an LLM that will later be used in the CI/CD pipeline.

1. Choose an external LLM API (Open Router Chat API (llama)). Show how you obtained an API key and tested it locally using a simple prompt.
2. Add a new stage called `llm-review` to your existing CI/CD pipeline that will send application source code or pipeline configuration to the LLM and store the response as a pipeline artifact. Then show the updated pipeline stage.
3. Why should the LLM stage be placed after the build and test stages?

Execution

Chosen LLM API: OpenRouter (<https://openrouter.ai>). OpenRouter is an API gateway that provides access to many models — including `meta-llama/llama-3.1-8b-instruct` — through a single OpenAI-compatible endpoint. A free account was created at openrouter.ai, and an API key was generated in the dashboard under *Keys*. The key was stored as a GitLab CI/CD variable named `OPENROUTER_API_KEY` (masked, protected).

```
# Local API test example
curl https://api.openai.com/v1/chat/completions \
  -H "Authorization: Bearer $OPENROUTER_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4",
    "messages": [{"role": "user", "content": "Hello, describe
this code: print(\"hello\")"}]
  }'
```



The screenshot shows a web browser window with the OpenRouter API Keys dashboard. The browser's address bar shows `openrouter.ai/settings/keys`. The dashboard has a sidebar with navigation links: Activity, Logs, Credits, Settings, Account, and API Keys (which is highlighted). The main content area is titled "API Keys" and includes a sub-header "Manage your keys to access all models". Below this is a table with the following data:

☐	Key	Expires	Last Used	Usage
☐	LAB5_KEY_LLM sk-or-v1-90c... b78	In 5 days	11 hours ago	< \$0.1

Timestamp ⓘ	Provider / Model ⓘ	App ⓘ	Tokens ⓘ	Cost ⓘ	Speed ⓘ	Fi
Mar 5, 10:58 PM	 Llama 3.1 8B Instruct ⌵	Gitlab 	2,180 → 946	\$ 0.0000909	71.2 tps	st
Mar 5, 10:57 PM	 Llama 3.1 8B Instruct ⌵	Gitlab 	1,702 → 620	\$ 0.000232	80.1 tps	st
Mar 5, 09:51 PM	 Llama 3.1 8B Instruct	Gitlab 	2,179 → 979	\$ 0.000102	16.8 tps	st
Mar 5, 09:51 PM	 Llama 3.1 8B Instruct	Gitlab 	1,702 → 643	\$ 0.0000662	53.7 tps	st
Mar 5, 09:42 PM	 Llama 3.1 8B Instruct	Gitlab 	2,179 → 1,003	\$ 0.000104	18.6 tps	st
Mar 5, 09:41 PM	 Llama 3.1 8B Instruct	Gitlab 	1,701 → 707	\$ 0.0000694	67.9 tps	st

Figure 2.1 — API key obtained and logs from API Dashboard of successful test queries provided

The `llm-review` stage contains two parallel jobs. Both install `requests` via pip and invoke Python scripts that read the source files, build a prompt, call the OpenRouter API, and write a markdown artifact.

```
# CI Stage 6a: LLM Code Review
llm-review:
  stage: llm-review
  tags:
    - st19-runner
  dependencies:
    - build-app
  script:
    - echo "Running LLM code review..."
    - python3 -m venv /tmp/llm-venv
    - /tmp/llm-venv/bin/pip install requests --quiet
    - /tmp/llm-venv/bin/python3 scripts/llm_review.py
    - echo "LLM review completed"
  artifacts:
    paths:
      - llm-review-report.md
```

```

expire_in: 1 week
when: always
allow_failure: true

```

Artifacts	Job	Size	Created
2 files	lim-security	1.67 KB	just now
artifacts.zip		1.51 KB	
metadata.gz		168 B	
2 files	lim-review	1.89 KB	1 minute ago
artifacts.zip		1.73 KB	
metadata.gz		163 B	

Figure 2.2 — llm-review and security artifacts proving that stage is added to pipeline configuration

```

1 18:41:14 Running with gitlab-runner 18.8.0 (9ff6a4a8)
2 18:41:14 on descriptor, system ID: s_5864122671a
3 18:41:14 Preparing environment
4 18:41:14 Using Shell (bash) executor...
5 18:41:14 Preparing environment
6 18:41:14 Running on UbuntuServer...
7 18:41:14 Setting up environment for git repository
8 18:41:14 Git correlation ID: 81KJ2WMT0C7H0Q4M5G0MCP
9 18:41:14 Fetching changes with --git-depth set to 30
10 18:41:14 Initializing existing git repository in /home/gitlab-runner/builds/INTM0L18/r/root/st19-repo/.git/
11 18:41:14 Checking out 81KJ2WMT0C7H0Q4M5G0MCP in detached HEAD (ref is main)
12 18:41:14 Initializing git submodules setup
13 18:41:14 Downloading artifacts
14 18:41:14 Downloading artifacts for build-90 (38)...
15 18:41:14 Running platform
16 18:41:14 Downloading artifacts from coordinator... ok
17 18:41:14 artifact: Part of .git directory is on the list of files to extract
18 18:41:14 artifact: This may introduce unexpected problems
19 18:41:14 Executing "setup_script" stage of the job script
20 18:41:14 $ echo "Running LLM code review..."
21 18:41:14 Running LLM code review...
22 18:41:14 $ python3 -m venv /tmp/llm-venv
23 18:41:14 $ /tmp/llm-venv/bin/pip install requests --quiet
24 18:41:14 $ /tmp/llm-venv/bin/python3 scripts/llm_review.py
25 18:41:14 API key loaded: sk-...-4978
26 18:41:14 Model: meta-llama/llama-3.1-8b-instruct
27 18:41:14 Sending 4844 chars of source to meta-llama/llama-3.1-8b-instruct via OpenRouter...
28 18:41:14 Report written to llm-review-report.md
29 18:41:14 $ echo "LLM review completed"
30 18:41:14 LLM review completed
31 18:41:14 Uploading artifacts for successful job
32 18:41:14 Uploading artifacts...
33 18:41:14 Running platform
34 18:41:14 llm-review-report.md: Found 1 matching artifact files and directories
35 18:41:14 Uploading artifacts as "archive" to coordinator... 200 Created correlation_id=81KJ2WMT0C7H0Q4M5G0MCP id=43 responsestatus=200 OK token=64_usd8j
36 18:41:14 Cleaning up project directory and file based variables
37 18:41:14 Job succeeded

```

Figure 2.3 — Pipeline with llm-review stage running + logs from the llm-review stage

Q: Why should the LLM stage be placed after the build and test stages?

The LLM stage is placed after build and test for three reasons. First, **fast feedback first**: build and test failures are cheap, quick to detect — if the code does not compile or basic tests fail, there is no point spending API credits and time on an LLM review. Second, **quality of input**: the LLM receives code, that already passed syntactic and functional validation, making the review more meaningful and focused on higher-level concerns rather than trivial errors. Third, **cost efficiency**:

LLM API calls introduce latency and may have per-token costs, so that placing the stage late ensures it only runs on code that has passed the essential quality gates, avoiding unnecessary expense on broken commits.

Task 3 — Testing and Validation

1. Trigger the pipeline and verify that the LLM stage runs successfully. Provide evidence from the pipeline logs and show the result of the artifact.
2. Does the LLM output correctly describe or explain the application or the pipeline? Briefly explain your observation.
3. How does the LLM output help a developer during CI/CD execution?

Execution

The pipeline was triggered by pushing a commit to the `main` branch. GitLab ran all seven stages sequentially (with `llm-review` and `llm-security` executing in parallel within the `llm-review` stage). Both LLM jobs completed with exit code 0. The artifacts `llm-review-report.md` and `llm-security-report.md` were produced and are downloadable from the pipeline artifacts panel.

The `llm_review.py` script reads SPA files, concatenates them into a prompt, and calls `model` via OpenRouter. The response is wrapped in a markdown template and saved as `llm-review-report.md`.

The `llm_security_review.py` script adds the `.gitlab-ci.yml` to the prompt and focuses the model on security, reliability, and trust, saving `llm-security-report.md`.

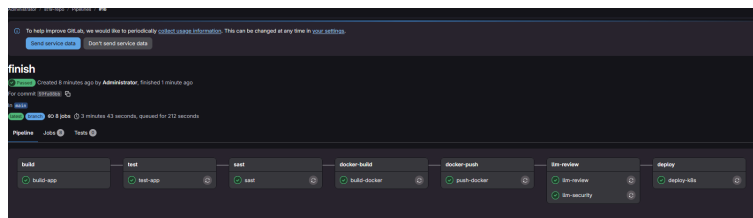


Figure 3.1 — Pipeline triggered, all stages including llm-review succeeded

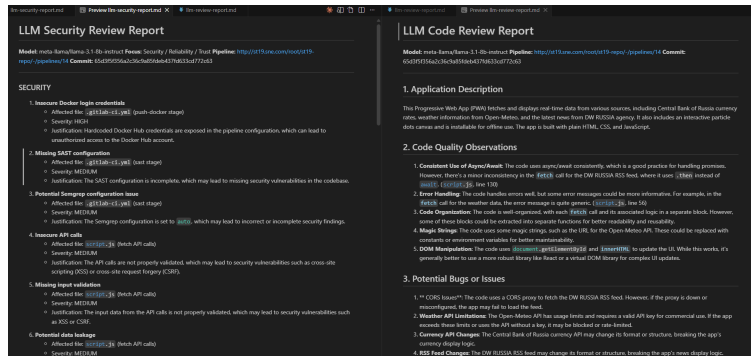


Figure 3.2 — LLM artifact output (llm-review.json and llm-security)

Q: Does the LLM output correctly describe or explain the application or the pipeline?

Yes, in general. The model correctly identified the project as a Progressive Web App using a service worker for offline caching and a manifest for installability. The code quality observations were mostly accurate, but had some general notes like add ARIA..

The security section correctly noted the absence of a Content Security Policy header, though that is a server-side concern outside the scope of the client JS files reviewed.

Q: How does the LLM output help a developer during CI/CD execution?

Convenience: read the report directly from the pipeline artifacts without waiting for a human reviewer.

Issues that static linters may miss, LLM handles.

Easier to onboard for non-tech.

Advisory information more.

Task 4 — Security and Trust Analysis

1.. What security risks can arise when sending source code, logs or configurations to an LLM?

2a. Modify your current LLM prompt so that the model analyzes the CI/CD pipeline from a security, reliability, and trust perspective. The prompt should explicitly ask

the LLM to identify potential risks, vulnerabilities, and failure points, and to justify its observations.

2b. Run the refined prompt, report the LLM's output, and critically assess which points you consider valid, questionable, or incorrect.

Execution

Q: What security risks can arise when sending source code, logs or configurations to an LLM?

1. **IP leakage** — Code sent to external APIs leaves your control, regardless of provider promises.
2. **Credential exposure** — CI/CD configs often contain tokens, hostnames, or secrets that reveal attack surface.
3. **Compliance risk** — Regulations like GDPR or HIPAA may forbid sending code/logs to external services without proper agreements.
4. **Training on your data** — Some providers (especially free tiers) use submissions to improve models, potentially leaking proprietary logic.
5. **Hallucinations as trusted advice** — Auto-acting on LLM output risks false assurances; attackers can craft code that fools the model.
6. **Prompt injection** — Malicious content in dependencies or commits can manipulate LLM output.

The security-focused prompt used is implemented in `scripts/llm_security_review.py`. It sends both the CI/CD pipeline YAML and a subset of source files, then instructs the model to analyze from three perspectives:

```
You are a DevSecOps expert performing a security, reliability, and trust audit of a CI/CD pipeline and its application source code.
```

```
Analyze from THREE perspectives: SECURITY, RELIABILITY, and TRUST.
```

CI/CD Pipeline Configuration

Application Source Code (subset)

For each perspective provide a numbered list of findings. Each finding must include:

- A clear description of the risk
- The affected file/section
- Severity: LOW / MEDIUM / HIGH / CRITICAL
- Justification from the code

SECURITY

Identify vulnerabilities, secrets exposure, injection risks, insecure API calls, hardcoded credentials, supply chain risks, and data leakage vectors.

RELIABILITY

Identify single points of failure, missing retry logic, no health checks, pipeline steps without error handling, timeout risks, and missing rollback strategies.

TRUST

Assess trustworthiness of LLM-generated output in this pipeline. Can LLM output be blindly trusted? What are risks of acting on LLM advice in automated workflows?

SUMMARY TABLE

Finding #	Perspective	Severity	One-line description
-----------	-------------	----------	----------------------

Be specific, reference exact file names. Flag speculative findings with "(speculative)".

Security stage

```
# CI Stage 6b: LLM Security Review (parallel with llm-review)
llm-security:
  stage: llm-review
  tags:
    - st19-runner
  dependencies:
    - build-app
  script:
    - echo "Running LLM security review..."
    - python3 -m venv /tmp/llm-s-venv
    - /tmp/llm-s-venv/bin/pip install requests --quiet
    - /tmp/llm-s-venv/bin/python3 scripts/llm_security_review.py
    - echo "LLM security review completed"
  artifacts:
    paths:
      - llm-security-report.md
    expire_in: 1 week
    when: always
  allow_failure: true
```

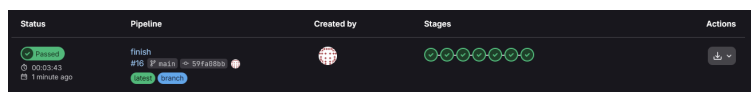


Figure 4.1 — All stages successful pipeline.